Gestionale Prenotazioni

Documentazione Tecnica Completa — v2.0

Spring Boot 4.0 · Java 21 · PostgreSQL · Docker · Cloudinary

Versione 2.0 | Aprile 2026

1. Panoramica del Progetto

Gestionale Prenotazioni (artifactId: formerjob) è un'applicazione backend RESTful sviluppata con Spring Boot 4.0 e Java 21. Gestisce eventi, biglietti, utenti e pagamenti con funzionalità di checkout multi-biglietto, reset password via email, upload immagini su Cloudinary e generazione PDF della documentazione OpenAPI.

1.1 Stack Tecnologico

Linguaggio: Java 21
Framework: Spring Boot 4.0.5
Database: PostgreSQL 16
ORM: Spring Data JPA / Hibernate
Sicurezza: Spring Security (profili dev/prod)
Email: Spring Boot Starter Mail — Mailtrap SMTP
Immagini: Cloudinary HTTP5 SDK v2.3.2
Documentazione: SpringDoc OpenAPI 2.8.14 (Swagger UI)
PDF: OpenPDF (LibrePDF) 1.3.40
Mapping: ModelMapper 3.2.6 + Lombok
Test: JUnit 5, Spring Security Test, Mockito
Build: Apache Maven (mvnw wrapper)
Container: Docker + Docker Compose (app, db, SonarQube)
Frontend CORS: http://localhost:4200 (Angular)

1.2 Struttura Package

- **Controller:** Endpoint REST; AbstractController<DTO> espone CRUD standard ereditato da tutti i controller
- **Service:** Logica di business; AbstractService<Entity,DTO> implementa ServiceDTO<DTO>
- **Repository:** Interfacce JPA Spring Data per accesso database
- **Model:** Entità JPA: User, Event, Ticket, Payment e relativi enum
- **Dto:** DTO di disaccoppiamento API/dominio: EventDto, TicketDto, PaymentDto, PurchaseOrderDto, TicketHolderDto…
- **Mapper:** Conversione Entity ↔ DTO tramite ModelMapper (AbstractMapper)
- **Security:** SecurityConfig (prod), DevSecurityConfig (dev), CommonConfig, CustomUserDetailsService
- **Exception:** UserNotFoundException, EventNotFoundException gestite da GlobalExceptionHandler

2. Modello dei Dati

2.1 Entità User

Tabella: formerjob."User". Rappresenta l'utente; il campo email è usato come username.

Campo	Tipo	Nullable	Descrizione
id	Integer	No	PK, generata IDENTITY
name	String	Sì	Nome
surname	String	Sì	Cognome
email	String	Sì	Email UNIQUE — username autenticazione
password	String	Sì	Password hashata
dateOfBirth	Date	Sì	Data di nascita
role	Role	Sì	Enum: ADMIN | USER
resetToken	String	Sì	Token per il reset password (UUID)
resetTokenExpiry	Date	Sì	Scadenza del reset token
tickets	List<Ticket>	Sì	OneToMany mappedBy="user"
payments	List<Payment>	Sì	OneToMany mappedBy="user"

2.2 Entità Event

Tabella: formerjob."Event". Rappresenta un evento con disponibilità di biglietti.

Campo	Tipo	Nullable	Descrizione
id	Integer	No	PK
name	String	No	Nome (max 100 car.)
description	String	No	Descrizione (max 300 car.)
location	String	No	Luogo (max 50 car.)
imagePath	String	No	URL immagine (max 100 car.)
date	Date	No	Data evento
maxTickets	Integer	No	Capienza massima (default 250)
selledTickets	Integer	No	Biglietti venduti (default 0)
ticketPrice	Double	No	Prezzo unitario
type	Type	Sì	Enum tipo evento
tickets	List<Ticket>	Sì	OneToMany cascade=REMOVE

2.3 Entità Ticket

Tabella: formerjob.ticket. Biglietto emesso per un utente. Ha ora relazione con Payment.

Campo	Tipo	Nullable	Descrizione
id	Integer	No	PK
name	String	No	Nome intestatario
surname	String	No	Cognome intestatario
price	Double	No	Prezzo biglietto
creationDate	LocalDateTime	No	Timestamp creazione
user	User (FK)	No	ManyToOne LAZY → user_id
event	Event (FK)	No	ManyToOne LAZY → event_id
payment	Payment (FK)	No	ManyToOne LAZY → payment_id [NUOVO]

2.4 Entità Payment

Tabella: formerjob.payment. Registra ogni transazione. Ora ha relazione OneToMany con Ticket.

Campo	Tipo	Nullable	Descrizione
id	`Integer`	No	PK
method	`PaymentMethod`	No	Enum: CREDIT_CARD | PAYPAL | BANK_TRANSFER
totalPrice	`double`	Sì	Importo totale
date	`LocalDateTime`	No	Data/ora transazione
user	`User (FK)`	No	ManyToOne LAZY → user_id
event	`Event (FK)`	No	ManyToOne LAZY → event_id
tickets	`List<Ticket>`	Sì	OneToMany mappedBy="payment" cascade=ALL [NUOVO]

2.5 Enum di Dominio

- **Type (tipo evento):** CONCERTI, TEATRO, SPORT, MOSTRA_E_MUSEI, ALTRE_MANIFESTAZIONI, EVENTI_INTERNAZIONALI, CINEMA
- **PaymentMethod:** CREDIT_CARD, PAYPAL, BANK_TRANSFER
- **Role:** ADMIN, USER

3. API REST — Riferimento Completo

Tutti i controller ereditano da AbstractController<DTO> che espone: GET /getall, GET /read?id=, POST /insert, PUT /update, DELETE /delete?id=.

3.1 Autenticazione | /auth

Metodo	Endpoint	Descrizione
POST	`/auth/register`	Registra nuovo utente. Body: RegisterRequestDto → LoginResponseDto (HTTP 201)
POST	`/auth/login`	Login. Body: LoginRequestDto → LoginResponseDto
POST	`/auth/forgot-password`	Invia email con link reset. Body: {"email":"…"}
POST	`/auth/reset-password`	Imposta nuova password. Body: {"token":"…","newPassword":"…"}

3.2 Utenti | /User

Metodo	Endpoint	Descrizione
GET	`/User/getall`	Tutti gli utenti
GET	`/User/read?id={id}`	Utente per ID
POST	`/User/insert`	Crea utente
PUT	`/User/update`	Aggiorna utente
DELETE	`/User/delete?id={id}`	Elimina utente
GET	`/User/findByName?name=`	Per nome (lista)
GET	`/User/findBySurname?surname=`	Per cognome (lista)
GET	`/User/findByEmail?email=`	Per email (singolo)
GET	`/User/findByNameAndSurname`	Per nome e cognome
GET	`/User/findBySurnameAndEmail`	Per cognome e email

3.3 Eventi | /Event

Metodo	Endpoint	Descrizione
GET	`/Event/getall`	Tutti gli eventi
GET	`/Event/read?id={id}`	Evento per ID
POST	`/Event/insert`	Crea evento (body: EventDto)
POST	`/Event/buildEventFromDto`	Crea evento e restituisce entità Event
PUT	`/Event/update`	Aggiorna evento
DELETE	`/Event/delete?id={id}`	Elimina evento
GET	`/Event/findByName?name=`	Per nome
GET	`/Event/findByDescription?description=`	Per descrizione
GET	`/Event/findByType?type=`	Per tipo (enum)
GET	`/Event/findByLocation?location=`	Per location
GET	`/Event/findByDataBetween`	Tra due date (startDate, endDate yyyy-MM-dd)
GET	`/Event/findByDataAfter?data=`	Dopo una data
GET	`/Event/findByDataBefore?data=`	Prima di una data
GET	`/Event/advancedSearch`	Ricerca multi-filtro (name, description, location, startDate, endDate)
GET	`/Event/findBySelledTickets?selledTickets=`	Uguale a N biglietti venduti
GET	`/Event/findBySelledTicketsLessThan?selledTickets=`	Meno di N biglietti venduti
GET	`/Event/findBySelledTicketsGreaterThan?selledTickets=`	Più di N biglietti venduti
GET	`/Event/findByTicketPrice?ticketPrice=`	Per prezzo biglietto
GET	`/Event/findTop5MostRemunerative`	Top 5 eventi più remunerativi

GET	/Event/getSelledTicketsByEventId?eventId=	Biglietti venduti per evento
GET	/Event/getAvailableTicketsByEventId?eventId=	Biglietti disponibili per evento
GET	/Event/findById?id=	Evento per ID (alias)

3.4 Biglietti | /Ticket

Metodo	Endpoint	Descrizione
GET	/Ticket/getall	Tutti i biglietti
GET	/Ticket/read?id={id}	Biglietto per ID
POST	/Ticket/insert	Crea biglietto
PUT	/Ticket/update	Aggiorna biglietto
DELETE	/Ticket/delete?id={id}	Elimina biglietto
GET	/Ticket/findByNameAndSurname	Per nome e cognome
GET	/Ticket/findTicketByCreationDate	Per data/ora creazione (ISO date-time)
GET	/Ticket/findTicketByUserId?userId=	Per ID utente [NUOVO]
GET	/Ticket/findTicketByPriceGreaterThanEqual?price=	Prezzo ≥ valore
GET	/Ticket/findTicketByPriceLessThanEqual?price=	Prezzo ≤ valore
GET	/Ticket/findTicketByPriceRange	Range prezzo (initialPrice, endPrice)
GET	/Ticket/findTicketByDateRange	Range temporale (startTime, endTime)

3.5 Pagamenti | /api/payment

Metodo	Endpoint	Descrizione
GET	/api/payment/getall	Tutti i pagamenti
GET	/api/payment/read?id={id}	Pagamento per ID
POST	/api/payment/insert	Inserisce pagamento
PUT	/api/payment/update	Aggiorna pagamento
DELETE	/api/payment/delete?id={id}	Elimina pagamento
GET	/api/payment/history	Storico completo pagamenti
GET	/api/payment/method/{method}	Filtra per metodo (PAYPAL, CREDIT_CARD, BANK_TRANSFER)
GET	/api/payment/user/{userId}	Storico pagamenti di un utente

3.6 Checkout | /api/checkout

Metodo	Endpoint	Descrizione
POST	/api/checkout	Acquisto singolo. Body: PaymentDto (include lista tickets come TicketHolderDto). Crea Payment+Ticket, scala disponibilità, invia email.

3.7 Purchase Order | /api/orders [NUOVO]

Endpoint per ordini multi-biglietto. Gestito da PurchaseOrderController/Service.

Metodo	Endpoint	Descrizione
POST	/api/orders	Crea ordine. Body: PurchaseOrderDto {payment: PaymentDto, tickets: List<TicketDto>}

3.8 Email | /email

Metodo	Endpoint	Descrizione
POST	/email/send-test-email	Invia email di conferma acquisto. Body: TicketDto
POST	/email/invia-mail-pdf?id={ticketId}	Invia email con PDF del biglietto allegato

		(AllegatoService)

3.9 Upload Immagini | /api/cloudinary

Metodo	Endpoint	Descrizione
POST	`/api/cloudinary/upload`	Carica immagine (multipart/form-data, campo "file"). Risposta: ImageUploadResponseDto {publicId, secureUrl}

3.10 Documentazione | /docs

Metodo	Endpoint	Descrizione
GET	`/docs/json`	Specifica OpenAPI in formato JSON
GET	`/docs/pdf`	Documentazione API come PDF scaricabile

Swagger UI: http://localhost:8080/swagger-ui/index.html

4. Data Transfer Objects (DTO)

4.1 LoginRequestDto

- email: String
- password: String

4.2 LoginResponseDto

- id: Integer
- name, surname, email: String
- role: Role (ADMIN | USER)
- status: boolean

4.3 RegisterRequestDto

- name, surname, email, password, taxCode: String
- dateOfBirth: Date

4.4 UserDto

- id: Integer; name, surname, email, password: String
- dateOfBirth: Date; role: Role
- ticketIds: List<Integer>; paymentIds: List<Integer>

4.5 EventDto

- id, maxTickets, selledTickets: Integer; name, description, location, imagePath: String
- date: Date; ticketPrice: Double; type: Type
- ticketIds: List<Integer>

4.6 TicketDto

- ticketId, userId, eventId, paymentId: Integer [paymentId è NUOVO]
- name, surname: String; price: Double; creationDate: LocalDateTime

4.7 TicketHolderDto [NUOVO]

Usato all'interno di PaymentDto per specificare nome e cognome di ogni intestatario del biglietto.

- name: String
- surname: String

4.8 PaymentDto

- id: Integer; method: PaymentMethod; totalPrice: Double
- date: LocalDateTime (ISO-8601); userId: Integer; eventId: Integer
- tickets: List<TicketHolderDto> [NUOVO — lista intestatari per checkout multi-biglietto]

4.9 PurchaseOrderDto [NUOVO]

Wrapper per ordini che contengono sia dati di pagamento che lista di biglietti completi.

- payment: PaymentDto — dati della transazione
- tickets: List<TicketDto> — biglietti da generare con nome, cognome e prezzo

4.10 ImageUploadResponseDto

- publicId: String — ID pubblico Cloudinary
- secureUrl: String — URL HTTPS immagine

5. Architettura e Pattern di Progetto

5.1 Template Method — Abstract Layer

AbstractController<DTO> espone CRUD standard delegando ad AbstractService<Entity,DTO>, che implementa ServiceDTO<DTO> e usa AbstractMapper<Entity,DTO>. I layer concreti aggiungono solo logica specifica del dominio.

```
AbstractController<DTO>  ← extends  EventController / TicketController / UserController / PaymentController
AbstractService<E,DTO>   ← extends  EventService / TicketService / UserService / PaymentService
AbstractMapper<E,DTO>    ← extends  EventMapper / TicketMapper / UserMapper / PaymentMapper
```

5.2 Gestione Eccezioni Globale

GlobalExceptionHandler (@ControllerAdvice) intercetta UserNotFoundException e EventNotFoundException restituendo HTTP 404.

5.3 Sicurezza — Profili Spring

- **Profilo dev:** dev: DevSecurityConfig — accesso libero a tutti gli endpoint (sviluppo/test). SQL visibile (show-sql=true).
- **Profilo prod:** prod: SecurityConfig — Spring Security attivo. SQL nascosto. sql.init.mode=never.

CustomUserDetailsService carica l'utente dal DB per email; CommonConfig contiene bean condivisi tra profili.

5.4 Reset Password

Flusso a due step gestito da UserService:

- POST /auth/forgot-password → genera UUID, salva in User.resetToken e resetTokenExpiry, invia email con link contenente il token
- POST /auth/reset-password → verifica token, controlla scadenza, aggiorna password hashata, cancella il token

5.5 Flusso Checkout (/api/checkout)

CheckoutService.completePurchase(PaymentDto) — per ogni TicketHolderDto nella lista:

- Recupera User ed Event tramite ID
- Verifica disponibilità (maxTickets − selledTickets > 0)
- Crea e persiste Payment
- Crea e persiste Ticket per ogni TicketHolderDto (con riferimento al Payment)
- Incrementa selledTickets dell'evento
- Invia email di conferma tramite EmailService

5.6 Purchase Order (/api/orders)

PurchaseOrderService.processOrder(PurchaseOrderDto) gestisce ordini composti: elabora payment e lista ticket come unità atomica, utile per acquisti multipli da un unico carrello.

5.7 Allegato Email PDF

AllegatoService.inviaOrdineConAllegato(ticket, event, user) genera un PDF del biglietto e lo allega all'email di conferma tramite MimeMessage (endpoint POST /email/invia-mail-pdf?id={ticketId}).

5.8 Generazione Documentazione PDF

SwaggerService.getApiDocsJson() recupera la specifica OpenAPI da /v3/api-docs; PdfService.generaPdfDaSwagger(json) la converte in PDF tramite OpenPDF, scaricabile via GET /docs/pdf.

6. Configurazione

6.1 application.properties

spring.datasource.url: jdbc:postgresql://localhost:5432/formerjob
spring.datasource.username: postgres
spring.datasource.password: root
spring.jpa.hibernate.ddl-auto: update
spring.sql.init.mode: always (never in prod)
spring.sql.init.data-locations: classpath:data.sql — dati di seed iniziali
server.port: 8080
spring.profiles.active: prod (cambiare in "dev" per sviluppo)
spring.mail.host: sandbox.smtp.mailtrap.io
spring.mail.port: 2525
app.mail.from: prova@gestionale.local
cloudinary.cloud_name: dstpobuge
cloudinary.api-key: 578733462843927
cloudinary.api-secret: FiwS5H_5RbyCXkQ8SuRlQhjcwLA

6.2 Profili Spring

- **Profilo dev:** dev: show-sql=true; DevSecurityConfig (tutto aperto); usato in Docker
- **Profilo prod:** prod: show-sql=false; SecurityConfig attivo; sql.init.mode=never

6.3 Dati Iniziali

src/main/resources/data.sql contiene INSERT per popolare le tabelle al primo avvio (profilo non-prod). Il file postman/seed-postman.sql offre dati aggiuntivi per i test Postman.

6.4 Schema Database

```
-- docker/postgres/init/01-create-schema.sql
CREATE SCHEMA IF NOT EXISTS formerjob;
```

Lo script viene eseguito automaticamente dal container PostgreSQL al primo avvio. Hibernate crea/aggiorna le tabelle con ddl-auto=update.

7. Deploy con Docker

7.1 Servizi docker-compose.yml

Servizio	Container	Porta	Note
app	formerjob_app	8080:8080	Spring Boot, profilo dev, attende db healthy. Build da Dockerfile multi-stage Maven+JRE21
db	formerjob_db	5432:5432	PostgreSQL 16, volume persistente, healthcheck pg_isready, esegue init SQL
sonarqube	sonarqube	9000:9000	Analisi qualità codice (SonarQube Community)
sonar_db	sonarqube_db	—	PostgreSQL 15 dedicato a SonarQube

7.2 Dockerfile — Multi-stage Build

Stage 1 (builder): maven:3.9.11-eclipse-temurin-21 — scarica dipendenze e compila il JAR (skipTests).

Stage 2 (runtime): eclipse-temurin:21-jre — copia solo il JAR finale, porta 8080 esposta.

7.3 Variabili d'Ambiente (app)

SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/formerjob
SPRING_DATASOURCE_USERNAME / PASSWORD: postgres / postgres
SPRING_PROFILES_ACTIVE: dev
SPRING_MAIL_*: configurazione Mailtrap passata come env
CLOUDINARY_*: cloud_name, api-key, api-secret

7.4 Comandi Principali

```
# Prima volta (build + avvio)
docker compose build app
docker compose up -d db app

# Aggiornamento dopo modifica codice
docker compose build app && docker compose up -d --no-deps app

# Log in tempo reale
docker compose logs -f app

# Stop conservando i dati
docker compose down

# Reset completo (elimina volumi DB)
docker compose down -v

# SonarQube
docker compose up -d sonarqube sonar_db
# http://localhost:9000  (admin/admin al primo accesso)
```

8. Testing

8.1 Struttura dei Test

- **EventServiceTest.java:** Test unitari per EventService (ricerche, filtri, logica biglietti)
- **EventControllerTest.java:** Test di integrazione per EventController (MockMvc)
- **TicketServiceTest.java:** Test unitari per TicketService (CRUD e ricerche)
- **TicketControllerTest.java:** Test di integrazione per TicketController
- **PaymentServiceTest.java:** Test unitari per PaymentService (filtri, storico, metodi)
- **PaymentControllerTest.java:** Test di integrazione per PaymentController
- **CheckoutServiceTest.java:** Test per il flusso checkout completo con email
- **UserServiceTest.java:** Test unitari per UserService (incl. reset password)
- **UserControllerTest.java:** Test di integrazione per UserController
- **FormerjobApplicationTests.java:** Smoke test caricamento contesto Spring

8.2 Esecuzione

```
./mvnw test                      # tutti i test
./mvnw test -pl . -Dtest=EventServiceTest  # test singolo
```

8.3 Collection Postman

postman/Gestionale-Prenotazioni.postman_collection.json contiene esempi per tutti gli endpoint. I file seed-postman.sql e test-email-data.sql permettono di popolare il DB con dati di test prima di eseguire le chiamate.

9. Avvio Rapido (Quick Start)

9.1 Prerequisiti

- Java 21 (JDK)
- Maven o mvnw wrapper incluso
- Docker Desktop (raccomandato) oppure PostgreSQL 16 locale
- Credenziali Mailtrap per test email (già configurate nel progetto)
- Account Cloudinary (già configurato nel progetto)

9.2 Avvio con Docker (metodo raccomandato)

```
docker compose build app
docker compose up -d db app
docker compose logs -f app   # aspetta "Started FormerjobApplication"

# Accesso:
Backend:  http://localhost:8080
Swagger:  http://localhost:8080/swagger-ui/index.html
Docs PDF: http://localhost:8080/docs/pdf
```

9.3 Avvio Locale (senza Docker)

```
# 1. Crea DB e schema
psql -U postgres -c "CREATE DATABASE formerjob;"
psql -U postgres -d formerjob -c "CREATE SCHEMA formerjob;"

# 2. Avvia (profilo prod)
./mvnw spring-boot:run

# Per profilo dev (sicurezza disabilitata, SQL visibile):
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev
```

9.4 Flusso di Lavoro Tipico

```
1. Modifica il codice
2. docker compose build app
3. docker compose up -d --no-deps app
4. Testa su Swagger UI o Postman
5. docker compose logs -f app  ← in caso di errori
```

10. Novità rispetto alla Versione Precedente

Riepilogo delle funzionalità aggiunte o modificate nell'aggiornamento del progetto:

- **User.resetToken / resetTokenExpiry:** Nuovi campi per il flusso di reset password via email
- **POST /auth/forgot-password:** Genera e invia token di reset all'email dell'utente
- **POST /auth/reset-password:** Aggiorna la password usando il token ricevuto via email
- **TicketHolderDto:** Nuovo DTO per specificare nome/cognome di ogni intestatario nel checkout
- **PaymentDto.tickets:** Campo aggiunto: lista di TicketHolderDto per checkout multi-biglietto
- **Ticket.payment (FK):** Il biglietto ora referenzia direttamente il pagamento (payment_id)
- **Payment.tickets (OneToMany):** Il pagamento ora ha una lista di biglietti associati
- **PurchaseOrderDto / Controller / Service:** Nuovo flusso /api/orders per ordini completi (payment + lista ticket)
- **GET /Ticket/findTicketByUserId:** Nuovo endpoint per filtrare ticket per ID utente
- **GET /Event/findById:** Alias per ricerca evento per ID
- **AllegatoService:** Servizio per invio email con PDF biglietto allegato (MIME multipart)
- **data.sql:** File di seed SQL integrato nelle risorse (eseguito all'avvio in profilo non-prod)

11. Glossario

- **DTO:** Data Transfer Object — classe per trasferire dati tra layer senza esporre entità JPA
- **JPA:** Jakarta Persistence API — standard Java per ORM
- **ORM:** Object-Relational Mapping — mappatura oggetti Java su tabelle DB
- **REST:** Representational State Transfer — stile architetturale per API HTTP
- **CORS:** Cross-Origin Resource Sharing — permette richieste da localhost:4200 (Angular)
- **Cloudinary:** Servizio cloud per archiviazione e gestione immagini/video
- **Mailtrap:** Servizio SMTP sandbox per test email in sviluppo (non invia email reali)
- **SonarQube:** Piattaforma open-source per analisi statica della qualità del codice
- **Swagger UI:** Interfaccia grafica per esplorare e testare le API REST
- **Checkout:** Flusso di acquisto: crea Payment e Ticket, aggiorna disponibilità, invia email
- **Purchase Order:** Ordine completo: incapsula PaymentDto + lista TicketDto per acquisti multipli
- **AllegatoService:** Service che genera PDF biglietto e lo allega all'email di conferma
- **resetToken:** UUID temporaneo salvato sull'utente per il recupero password, con scadenza